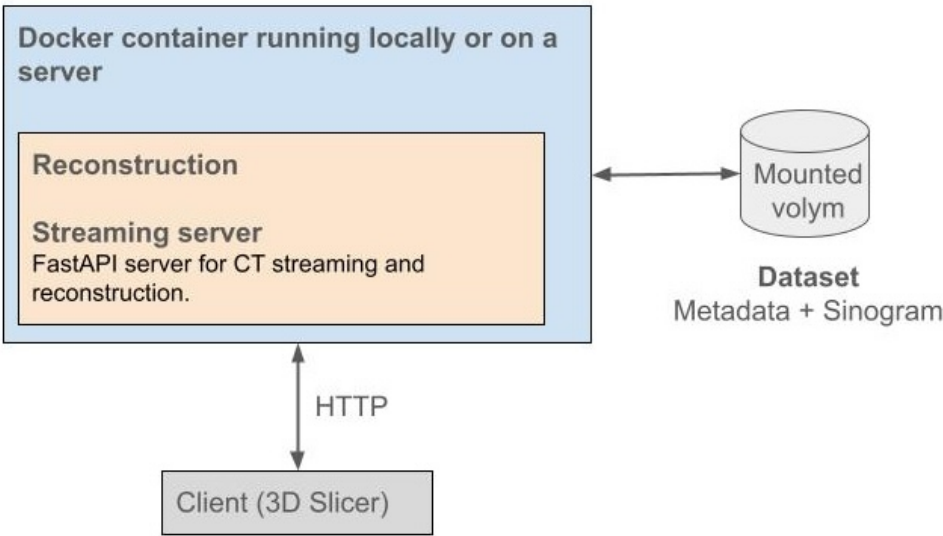

CT Sinogram Visualizer

SinoReconsVisual2 User Guide

Interactive inspection of CT acquisition geometry, projection data, regions of interest, and remote reconstruction in 3D Slicer

Pipeline Architecture: Client (3D Slicer) with FastAPI Streaming and Reconstruction Server



Project	CT_GUI_Docker
Guide version	1.0
Software tested with	3D Slicer 5.12.3
Date	August 2026

About this guide

This guide explains how to install, start, and use the CT-sinogram visualizer. The system consists of a CUDA-enabled reconstruction and streaming server and the SinoReconsVisual2 module for 3D Slicer. It is intended for researchers and CT operators who need to inspect raw projection data together with the physical acquisition geometry before reconstruction.

The current application targets wood CT, where internal features such as knots, cracks, density variation, and growth-related structures are studied. The same workflow can be used in other CT applications when compatible projection data and acquisition metadata are available.

Development status

The software is under active development. Interface details and backend behavior may change between revisions. Preserve important ROI definitions and reconstructed volumes outside temporary container storage.

Contents

1	System overview	1
2	Quick start for an existing server	1
3	Server installation and startup	2
3.1	Requirements	2
3.2	Download and build	2
3.3	Prepare the dataset	3
3.4	Start the server	3
3.5	Verify connectivity	4
4	Install the 3D Slicer module	4
5	Interface tour	5
5.1	Connect to server	5
5.2	Sinogram controls	5
5.3	Regions of interest	5
5.3.1	Safe ROI editing	7
5.3.2	Projection range	7
6	Run a reconstruction	8
6.1	Reconstruction methods	8
7	Interpret the views and logs	8
8	Troubleshooting	9

9	Operational recommendations	10
10	Current scope and limitations	10
A	API summary	10
B	Maintenance commands	10
C	References and acknowledgements	11

1 System overview

Raw CT projections are often stored as large numerical arrays that are difficult to inspect directly. This system links each selected projection to its source position, detector surface, rays, and scan trajectory. A client-server design keeps the 3D Slicer interface responsive while the backend handles large datasets and GPU reconstruction.

Pipeline Architecture: Client (3D Slicer) with FastAPI Streaming and Reconstruction Server

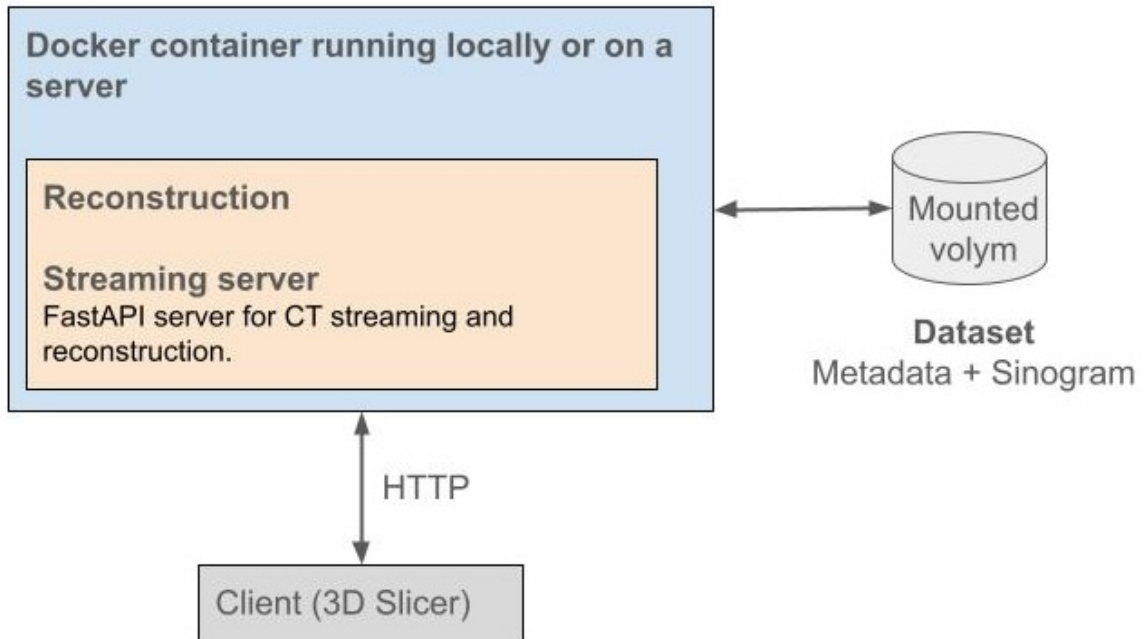


Figure 1: System architecture. 3D Slicer provides the user interface; the Docker/FastAPI backend loads the data, prepares acquisition geometry, streams sinogram slices, and runs reconstructions.

Component	Responsibility
3D Slicer client	Displays acquisition geometry and projections, defines regions of interest (ROIs), submits reconstruction jobs, and loads the returned NRRD volume.
SinoReconsVisual2 module	Implements the Slicer interface and communicates with the backend over HTTP.
FastAPI server	Enumerates samples, selects the active sample, serves geometry and projection slices, and starts reconstructions.
Docker container	Provides a reproducible CUDA, ASTRA, ODL, Python, VTK, and FastAPI environment.
Dataset directory	Stores the sample list, sinogram, trajectory arrays, shifts, and reconstruction metadata. It is normally mounted read-only.

2 Quick start for an existing server

Use this section when a server administrator has already started the backend and supplied a URL.

1. Start 3D Slicer and select **SinoReconsVisual2** from the module selector.
2. Enter the backend address in **Server HOST / IP**, including the protocol and port, for example `http://server.example.org:8000`.
3. Click **Connect to Server**. Continue only after the status changes to green **Connected**.
4. Choose a scan from **Select Sample**, then click **Load Sample**. Loading is complete when the metadata table, sinogram controls, geometry, and ROI controls become active.
5. Move the **Sinogram** slider and verify that the detector image and 3D source/detector geometry update together.
6. Add or select an ROI, set its physical bounds and projection range, then save it.
7. In **Reconstruction**, select the ROI, choose a reconstruction method, and click **Run reconstruction**.
8. After the returned NRRD is loaded, inspect it in the slice views and save it with Slicer's **Save Data** command.

Recommended first run

Use a modest ROI and resolution, retain the full projection range, and choose **adjoint** for a basic connectivity test or **fbp** for a quick reconstructed image. Use **landweber** only after the complete pipeline is known to work.

3 Server installation and startup

3.1 Requirements

The server host requires:

- 64-bit Linux with an NVIDIA CUDA-capable GPU;
- an NVIDIA driver compatible with CUDA 12.1;
- Docker Engine and NVIDIA Container Toolkit;
- Git and sufficient disk space for the image, datasets, cache, and outputs;
- network access from the Slicer computer to TCP port 8000, or an SSH tunnel.

Confirm GPU access before building:

```
nvidia-smi
docker run --rm --gpus all \
  nvidia/cuda:12.1.1-base-ubuntu20.04 nvidia-smi
```

3.2 Download and build

```
git clone https://github.com/ouaburst/CT_GUI_Docker.git
cd CT_GUI_Docker
docker build -t woodscan:cuda121 .
docker image inspect woodscan:cuda121
```

The first build downloads the CUDA base image and Python/Conda dependencies and may take considerable time.

3.3 Prepare the dataset

The directory mounted as `/samples_directory` must contain `samples.json` and one directory per sample:

```
/data/mito_samples/  
|-- samples.json  
'-- pine_16_1/  
    |-- sinogram.npy  
    |-- angles.npy  
    |-- axial_positions.npy  
    |-- shifts.npy  
    '-- metadata.json
```

Example `samples.json`:

```
[  
  {"folder": "pine_16_1", "name": "Pine 16, Disk 1"}  
]
```

The `folder` value must exactly match a child directory of the mounted dataset directory.

3.4 Start the server

Replace the dataset path with the absolute path on the server host:

```
docker run --rm -it \  
  --name woodscan-server \  
  --gpus all \  
  -p 8000:8000 \  
  -v /data/mito_samples:/samples_directory:ro \  
  woodscan:cuda121 \  
  python -m uvicorn odl_stream_server:app \  
  --host 0.0.0.0 --port 8000
```

Leave the terminal open and wait for **Application startup complete**. The server loads the first configured sample and precomputes geometry and sinogram cache information during startup.

```

docker run --rm -it -p 8000:8000 --gpus all -v /media/Store-SSD:/media/Store-SSD:ro --mount type=bind,source=/media/Store-SSD/real_datasets/ml_ready,target=/samples_directory,readonly woodscan:cuda121 python -m uvicorn odl
_stream_server:app --host 0.0.0.0 --port 8000

-- CUDA --

CUDA Version 12.1.1

Container image Copyright (c) 2016-2023, NVIDIA CORPORATION & AFFILIATES. All rights reserved.

This container image and its contents are governed by the NVIDIA Deep Learning Container License.
By pulling and using the container, you accept the terms and conditions of this license:
https://developer.nvidia.com/ngc/nvidia-deep-learning-container-license

A copy of this license is made available in this container at /NGC-DL-CONTAINER-LICENSE for your convenience.

[INFO] Started server process [1]
[INFO] Waiting for application startup.
[INFO] Selecting sample Pine 16, Disk 1 (/samples_directory/pine_16.1)...
[INFO] Loading data from: /samples_directory/pine_16.1
[INFO] Sinogram min -0.7572905421257019 max 35.488868713378906
[INFO] Sinogram loaded with shape (11066, 1560, 230) and dtype float32 in 0.00164 s
[INFO] Computed average pitch: 101.28 mm

[CHECK] Verifying reconstruction volume dimensions...
+ Width: 256.00 mm (expected: 210.00 mm)
+ Height: 256.00 mm (expected: 220.00 mm)
! Dimension mismatch!
Center of volume: (25.00, -90.00)
Creating odl geometry took 0.00205 seconds
[INFO] Precomputing and saving full geometry JSON to disk...
[DEBUG] Generating sensor geometry took 0.14269161224365234 seconds
[DEBUG] Saving geometry and trajectory took 0.07678723335266113 seconds
[INFO] Full geometry saved to /workspace/output/full_geometry.rpz
[INFO] Startup took 0.2564516067904883 seconds
[INFO] Application startup complete.
[INFO] Generating sinogram cache (jp2)
[INFO] Uvicorn running on http://0.0.0.0:8000 (Press CTRL+C to quit)
[INFO] Generated sinogram cache in 7.406380653381348 seconds (252196823 b)
Info: GET http://69398.research.ltu.se:8000/samples.json took 0.00258 s
INFO: 81.170.242.201:63815 - "GET /samples.json HTTP/1.1" 200 OK
Info: GET http://69398.research.ltu.se:8000/reconstruction_methods.json took 0.00141 s
INFO: 81.170.242.201:63815 - "GET /reconstruction_methods.json HTTP/1.1" 200 OK
WARNING: Invalid HTTP request received.
Info: GET http://130.240.244.73:8000/ took 0.00213 s
INFO: 91.230.168.245:54241 - "GET / HTTP/1.1" 404 OK
Info: GET http://130.240.244.73:8000/favicon.ico took 0.000455 s
INFO: 91.230.168.245:59807 - "GET /favicon.ico HTTP/1.1" 404 Not Found

```

Figure 2: Example server startup output. Successful startup ends with Uvicorn listening on port 8000; subsequent HTTP 200 entries indicate successful client requests.

For background operation, replace `-it` with `-d`. Then use:

```

docker logs -f woodscan-server
docker stop woodscan-server

```

3.5 Verify connectivity

```

curl http://localhost:8000/
curl http://localhost:8000/samples.json
curl http://localhost:8000/reconstruction_methods.json

```

Interactive API documentation is available at <http://localhost:8000/docs>. For a remote server, use its hostname or IP address. An SSH tunnel avoids exposing port 8000 directly:

```

ssh -L 8000:localhost:8000 USER@SERVER

```

With the tunnel active, enter `http://localhost:8000` in Slicer.

4 Install the 3D Slicer module

1. Install and start 3D Slicer 5.12.3.
2. Open **Edit > Application Settings > Modules**.
3. Under **Additional module paths**, add the absolute path to:

```

CT_GUI_Docker/3Dslicer/CT-Wood/CTWood/SinoReconsVisual2

```

4. Accept the settings and restart Slicer.
5. Select **Utilities > SinoReconsVisual2**, or choose the module from the module selector.

On first use, the module may install `pynrrd`, `pathvalidate`, or other small Python packages into Slicer's Python environment. Internet access may be needed.

Avoid loading the wrong checkout

If several copies of the repository exist, Slicer may load an older module. The Python console prints the paths of the reconstruction-method UI files at startup. Compare those paths with the checkout you intend to use and correct **Additional module paths** if necessary.

5 Interface tour

5.1 Connect to server

Control	Purpose
Server HOST / IP	Backend base URL, including <code>http://</code> or <code>https://</code> and the port.
Connect to Server	Downloads the sample list and reconstruction-method configuration.
Select Sample	Chooses the scan that the backend should make active.
Load Sample	Requests the selected sample, downloads compact geometry, displays metadata, and activates the remaining controls.
Sample metadata	Shows reconstruction bounds, dimensions, projection limits, and other scan-specific values supplied by the backend.

5.2 Sinogram controls

The index slider selects one projection. **Play** advances through the projection sequence, and the speed control changes the playback rate. While dragging, a preview may be shown first; the full-detail projection is loaded after the slider stops.

- **Show source and detector:** displays the source, detector, and associated rays for the current projection.
- **Show sinogram on sensor:** maps the current projection onto the detector surface in the 3D view.

Use these controls to locate abrupt intensity changes, missing projections, detector artifacts, inconsistent geometry, or unexpected motion along the trajectory.

5.3 Regions of interest

Click **+** to create an ROI. Select an entry in the list to edit it; click **-** to delete it. A trailing asterisk indicates unsaved changes. The ROI is a physical box in Slicer's RAS coordinate system:

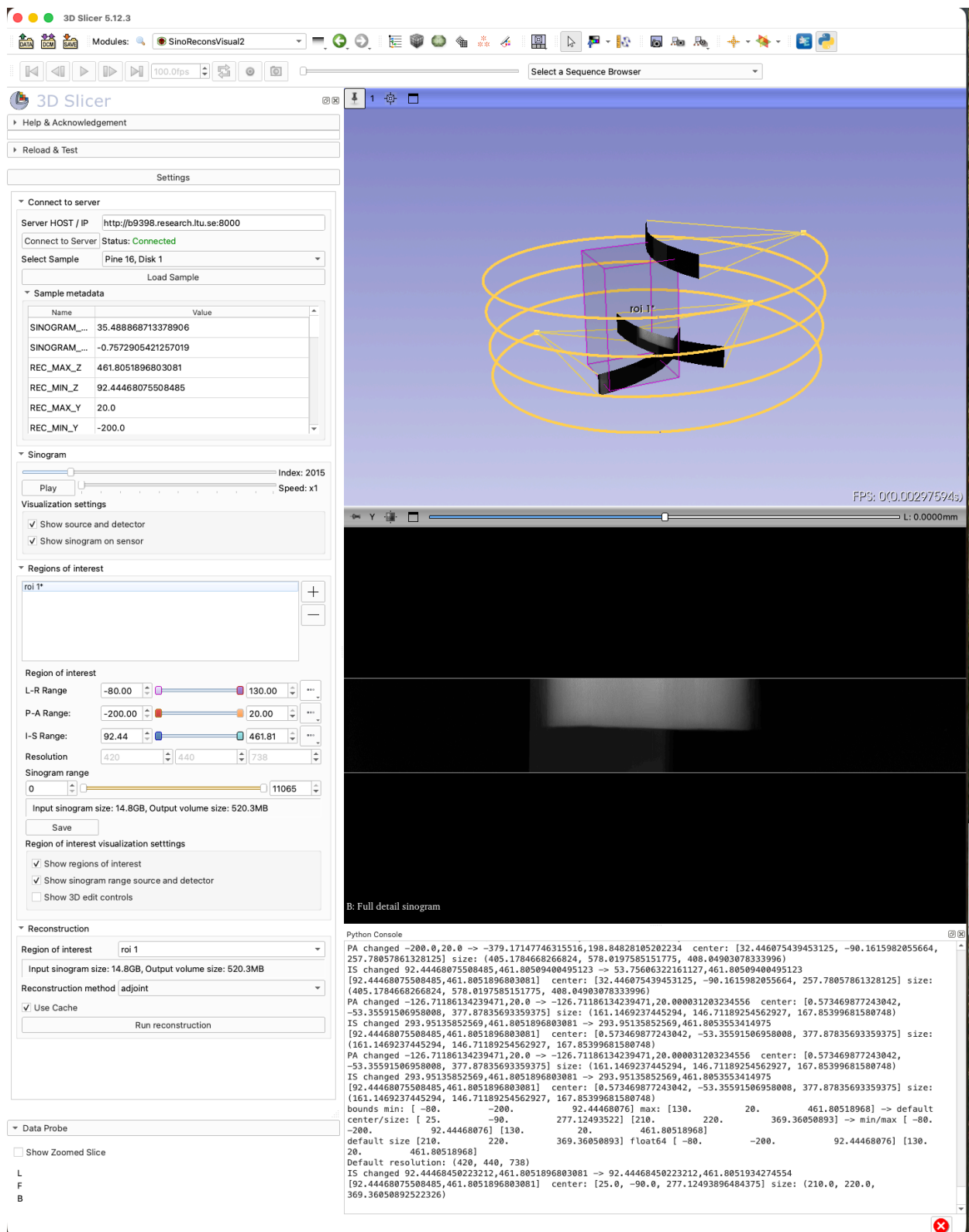


Figure 3: The SinoReconsVisual2 interface in 3D Slicer. The module controls are on the left, acquisition geometry appears in the upper-right 3D view, the selected full-detail projection is below, and the Python console is open at lower right.

Control	Meaning
L–R Range	Minimum and maximum left-right coordinates.
P–A Range	Minimum and maximum posterior-anterior coordinates.
I–S Range	Minimum and maximum inferior-superior coordinates.
Resolution	Number of output voxels in the three axes, not the physical size in millimeters.
Sinogram range	First and last projection indices used for this ROI.
Save	Stores the ROI definition as JSON for the active sample.
Input/output sizes	Estimated sinogram input size and float32 reconstructed-volume size.

ROI files are stored under:

```
~/Documents/SinoRecons/<sample-folder>/<roi-name>.json
```

Right-click an ROI for **Rename**, **Save**, **Open containing folder**, **Discard changes**, **Auto Resolution**, and **Delete**.

5.3.1 Safe ROI editing

When a sample is loaded, the metadata defines valid reconstruction bounds. For the sample shown in Figure 3, the nominal bounds are:

Axis	Minimum (mm)	Maximum (mm)
L–R	-80.00	130.00
P–A	-200.00	20.00
I–S	92.44	461.81

The numeric widgets are the safest way to define a precise crop. The 3D handles are convenient for exploration, but the current implementation does not strictly clamp a box manipulated in the 3D view to the metadata bounds.

Auto Resolution limitation

In the current implementation, **Auto Resolution** is enabled for a new ROI but is not recalculated after every ROI size change. A small crop may therefore retain the whole-volume resolution and unnecessarily large memory estimate. Right-click the ROI, disable **Auto Resolution**, and enter suitable voxel counts manually.

For a desired isotropic voxel spacing s , estimate each voxel count with

$$N_i \approx \text{round} \left(\frac{\text{ROI size}_i}{s} \right).$$

For example, a $161 \times 147 \times 168$ mm ROI at approximately 0.5 mm spacing requires about $322 \times 294 \times 336$ voxels. A float32 output of that size uses approximately 121 MiB before additional processing overhead.

5.3.2 Projection range

The full projection range provides the most complete angular and axial coverage. A smaller range reduces input and computation but may create limited-angle artifacts or exclude parts of the helical

trajectory. Use the orange start/end geometry display to relate the selected projection interval to the ROI.

6 Run a reconstruction

1. Select the intended ROI from the **Region of interest** list in the **Reconstruction** section.
2. Check the displayed input and output size estimates.
3. Choose a reconstruction method and configure its parameters.
4. Leave **Use Cache** enabled for repeated work unless the backend reports memory or cache errors.
5. Click **Run reconstruction** and confirm the dialog.
6. Wait for the server to finish. Canceling the Slicer progress dialog currently does not necessarily cancel the backend process.
7. When the NRRD is returned, verify that it appears in the slice views and volume rendering.
8. Save the result using Slicer's **Save Data** command.

6.1 Reconstruction methods

Method	Parameters	Recommended use
Adjoint	None	Simple unfiltered backprojection. Useful as a first end-to-end test; the result is normally blurred compared with FBP.
FBP	Filter type, frequency scaling, padding	Fast analytical reconstruction. This is the normal starting point for a quick usable volume.
Landweber	Iterations, relaxation	Iterative gradient reconstruction. It can be slow and sensitive to parameter choice. Use only after smaller methods work and monitor server GPU memory and logs.

For FBP, start with the server defaults: Ram-Lak filter, frequency scaling 1.0, and padding enabled. For Landweber, begin with a small ROI and conservative iteration count. The currently checked-in client source notes that Landweber could not be fully tested because of an ODL exception, so treat it as experimental.

7 Interpret the views and logs

The upper-right 3D view combines the scan trajectory, detector surfaces, rays, and ROI. The lower view shows the selected projection in full detail. Together they answer two complementary questions: where was a measurement acquired, and what did the detector measure at that position?

The Python console is useful during troubleshooting. Typical messages include:

- **Selecting sample** and **Loaded full geometry**: successful sample initialization;
- **Download (full detail)** and **Decoding image**: successful projection streaming;

- **LR changed, PA changed, and IS changed:** verbose ROI debug output generated while moving or resizing the box;
- **Origin and Spacing:** placement of a completed reconstruction in Slicer;
- **HTTP 200 entries on the server:** successful requests.

The repeated ROI messages are not, by themselves, errors. However, coordinate values far outside the sample metadata indicate that the 3D ROI was moved or scaled beyond the supported reconstruction volume. Restore a saved ROI with **Discard changes**, or enter valid ranges numerically.

8 Troubleshooting

Symptom	Action
Status remains disconnected	Verify the complete URL, test <code>/samples.json</code> in a browser or with curl, check the firewall, and confirm the container publishes port 8000.
Connection refused with localhost	Use localhost only when the server is on the Slicer computer or an SSH tunnel is active. Otherwise use the server hostname or IP.
Sample list missing	Ensure the mounted directory contains a file named exactly <code>samples.json</code> . Mount the parent dataset directory, not a single sample directory.
Sample fails to load	Confirm that all files listed in Section 3.3 exist, then inspect <code>docker logs woodscan-server</code> .
Docker cannot select NVIDIA runtime	Install or repair NVIDIA Container Toolkit, restart Docker, and repeat the CUDA container test.
CUDA out of memory	Stop competing GPU work, reduce ROI resolution, use a smaller ROI, disable cache if appropriate, or expose a GPU with more free memory.
Output estimate remains very large after cropping	Disable ROI Auto Resolution and enter voxel counts that match the crop.
ROI coordinates grow beyond metadata bounds	Stop using the 3D handles for that edit, restore a saved ROI or enter bounded numeric ranges.
Console fills with ROI messages	These are debug prints. Close the Python console for normal use; inspect it only when diagnosing a problem.
Changes to module code do not appear	Check the UI-file paths printed during module startup and remove stale entries from Slicer's Additional module paths . Restart or reload the module.
Reconstruction progress appears stuck at 99%	The client waits for the asynchronous server request and does not receive continuous progress for every method. Check the server terminal for activity.
Landweber fails	Reproduce with a small ROI, capture client and server logs, and use adjoint or FBP while the ODL issue is investigated.

9 Operational recommendations

- Begin with a small reconstruction to validate the server URL, dataset, GPU, and returned-volume placement.
- Name ROIs descriptively, for example `knot_upper`, `crack_01`, or `full_disk_lowres`.
- Save each ROI before reconstruction so that its center, size, projection range, and resolution are reproducible.
- Record the sample, ROI file, method, parameters, server revision, and Slicer version when producing research results.
- Keep the dataset mount read-only during routine operation.
- Treat input and output size labels as estimates; reconstruction libraries require additional RAM and GPU memory.
- Do not close the server terminal during an interactive `-it` session.
- Save final NRRD volumes to durable host storage. Container-local outputs disappear when a container started with `-rm` exits.

10 Current scope and limitations

The current data path is designed around ODL cone-beam geometry and has primarily been tested with a curved detector and square detector pixels. It is not a universal scanner interface: new scanner formats require a geometry/data-loader adapter that supplies compatible source positions, detector configuration, projection indexing, and reconstruction metadata.

Additional detector types, projection geometries, coverage visualization, and broader scanner adapters are natural extensions. Results should therefore be validated against known phantoms and scanner-specific expectations before quantitative use.

A API summary

Endpoint	Purpose
GET /	Health/status and basic usage hints.
GET /samples.json	Available sample names and folders.
GET /reconstruction_methods.json	Reconstruction methods and configurable parameters.
POST /select_sample	Makes a configured sample active.
GET /full_geometry_npz	Compact acquisition geometry used by the Slicer client.
GET /get_sinogram_slice/{index}	Full-detail projection.
GET /get_sinogram_slice_fast/{index}	Compressed preview projection.
POST /run_reconstruction	Starts a reconstruction from the supplied sample, ROI, projection range, method, and parameters.
GET /docs	Interactive Swagger/OpenAPI interface.

B Maintenance commands

Update and rebuild the server image:

```
git pull
docker build -t woodscan:cuda121 .
```

Inspect the running service:

```
docker ps
docker logs -f woodscan-server
nvidia-smi
curl http://localhost:8000/
```

C References and acknowledgements

- Fedorov, A. et al. (2012). 3D Slicer as an image computing platform for the Quantitative Imaging Network. *Magnetic Resonance Imaging*, 30, 1323–1341.
- Kak, A. C. and Slaney, M. (1988). *Principles of Computerized Tomographic Imaging*. IEEE Press.
- Slicer Community. 3D Slicer documentation and software platform: <https://www.slicer.org/>.
- Burström, O., Häger, J., and Öktem, O. (2026). CT-sinogram visualizer and CT_GUI_Docker software repository: https://github.com/ouaburst/CT_GUI_Docker.

The software was developed through collaboration involving Luleå University of Technology, KTH Royal Institute of Technology, and InfraVis. Development support was provided through InfraVis user support project T-2511-2307.